

Colorizing the Prokudin-Gorskii Glass Plates

Brennan Birn

CS 280A · Project 1 · UC Berkeley · Fall 2026

[Home](#)

Live page: <https://brennanbirn.github.io/project1/> | [GitHub repo](#)

One glass plate, three filtered exposures, and one reconstructed color photograph.

In 1907, Sergei Prokudin-Gorskii photographed scenes through blue, green, and red filters in sequence. The three exposures were recorded on a single glass plate. The Library of Congress later digitized these negatives, so each scan appears as a tall grayscale image containing three vertically stacked photographs.



A digitized plate, its three filter exposures, and the reconstructed color photograph.

Reconstructing a color image requires splitting each plate into its blue, green, and red exposures, then aligning the green and red channels to blue. I built the project in three stages:

1. A brute-force displacement search for the small JPEG plates.
2. A coarse-to-fine image pyramid for the larger TIFF plates.
3. A gradient-based pyramid that matches structure rather than raw brightness.

The final pyramid reduces the largest search from roughly 130,000 full-resolution candidate shifts per channel to 1,086 local comparisons across the pyramid. More importantly, matching Sobel gradient magnitudes fixes the one plate that raw intensity matching cannot: *emir*. That failure turned out not to be a matter of choosing NCC over SSD, but of deciding what image information should be matched in the first place.

The final pipeline registers all 18 plates correctly.

01 · Brute-force alignment on small plates

I use the blue exposure as the fixed reference. For each green and red channel, I test every integer displacement $(\Delta y, \Delta x)$ inside a search window, score the alignment, and retain the best result.

A positive vertical displacement moves an image down; a positive horizontal displacement moves it right. This follows NumPy indexing, where the origin is at the upper-left corner.

Similarity is evaluated only over the central 80% of the image. This avoids two sources of error:

- `np.roll` wraps pixels from one edge of the image to the other.
- The plate boundaries are scratched, darkened, and inconsistent across channels.

I tested two similarity metrics.

Sum of squared differences

SSD is minimized:

$$\text{SSD}(I_1, I_2) = \sum_{p \in \Omega} (I_1(p) - I_2(p))^2,$$

where Ω is the cropped comparison region.

Normalized cross-correlation

NCC is maximized:

$$\text{NCC}(I_1, I_2) = \frac{\sum_{p \in \Omega} (I_1(p) - \mu_1) (I_2(p) - \mu_2)}{\sqrt{\sum_{p \in \Omega} (I_1(p) - \mu_1)^2} \sqrt{\sum_{p \in \Omega} (I_2(p) - \mu_2)^2}}.$$

The mean-centered form shows why NCC is useful. If two images differ by a global affine brightness transformation,

$$I_2 = aI_1 + b,$$

the additive term b vanishes after mean centering, while the multiplicative term a cancels between the numerator and denominator. NCC is therefore insensitive to one global brightness scale and offset. Later, *emir* shows the limit of that assumption.

On the three small JPEG plates, SSD and NCC produced identical offsets.

Plate	Green offset	Red offset
Cathedral	(5, 2)	(12, 3)

Plate	Green offset	Red offset
Monastery	(-3, 2)	(3, 2)
Tobolsk	(3, 3)	(6, 3)

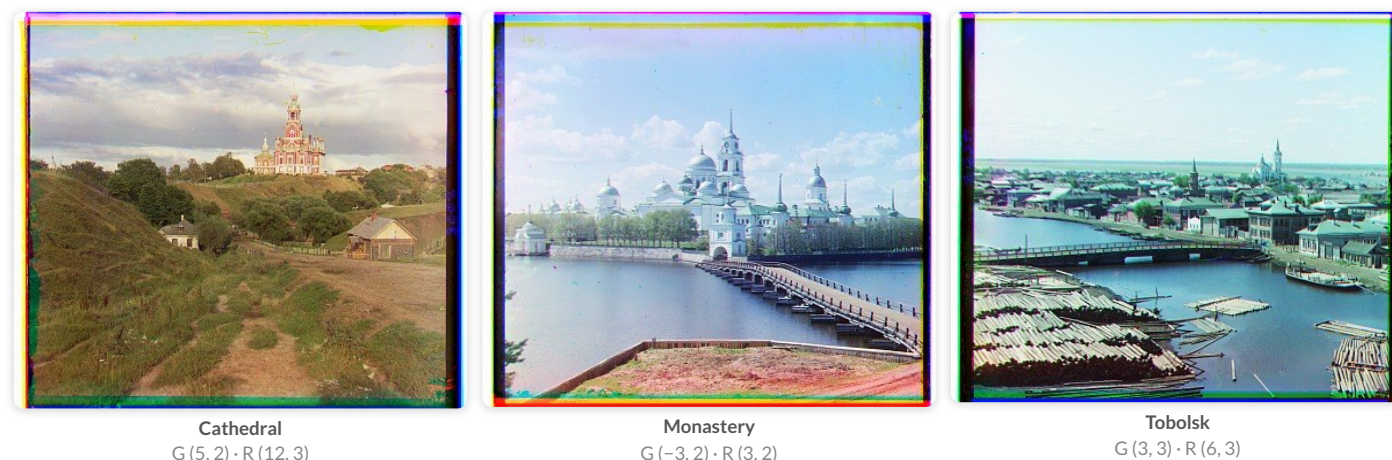


Figure 1. Single-scale search on the three JPEG plates. SSD and NCC produce the same offsets. Rooflines are a useful visual check because even small misregistrations appear quickly as red-blue ghosting along high-contrast edges.

Each alignment takes roughly 0.2 seconds.

A numerical bug worth avoiding

Images must be converted to floating point before any arithmetic. With `uint8`, negative values wrap around:

$$0 - 1 = 255$$

rather than

$$0 - 1 = -1.$$

This is especially harmful for SSD. A wrapped difference of 255 becomes

$$255^2 = 65,025,$$

while the correct squared difference should be

$$(-1)^2 = 1.$$

Since SSD is minimized, unsigned overflow can make a poor alignment appear artificially favorable. This is not random numerical noise; it creates a systematic bias toward incorrect shifts.

02 · The image pyramid

The full-resolution TIFF plates require displacements as large as 178 pixels. A brute-force search with half-width d evaluates

$$(2d + 1)^2$$

candidate shifts per channel. For $d = 180$, that becomes

$$(2(180) + 1)^2 = 130,321$$

candidates. Each comparison operates on an image containing roughly 11.6 million pixels, so an exhaustive full-resolution search is too expensive.

The solution is a coarse-to-fine image pyramid. I first estimate the displacement on a small, downsampled image, then propagate that estimate upward while refining locally at each larger scale.

Let $\mathbf{d}_\ell$ be the displacement at pyramid level ℓ , with $\ell = 0$ representing the full-resolution image. When each level is downsampled by two,

$$\mathbf{d}_\ell = 2\mathbf{d}_{\ell+1} + \delta_\ell,$$

where δ_ℓ is a small local refinement at level ℓ .

The recursive algorithm is:

1. If the shorter image dimension is below `min_size`, perform a brute-force search and return its displacement.
2. Otherwise, Gaussian-blur and downsample both images by a factor of two.
3. Estimate the lower-resolution displacement recursively.
4. Double that displacement to return to the current resolution.
5. Search a small window around the doubled estimate.

The multiplication in Step 4 is easy to omit. A displacement measured at half resolution must be doubled before it can be used at the next pyramid level. If the factor of two is missing, the final displacement is too small by

$$2^L,$$

where L is the number of pyramid levels.

On a two-level JPEG pyramid, that error is only a factor of four and can still look superficially plausible. On a five-level TIFF pyramid, it becomes a factor of 32: a true 178-pixel displacement can return as roughly 6 pixels.

Why blur before downsampling

Downsampling by two halves the Nyquist frequency. Frequencies above the new limit do not disappear when every other pixel is discarded; they alias into lower frequencies and create false structure.

Gaussian filtering removes those high-frequency components before decimation. The downsampled image then contains only structure that can be represented at the new resolution.

Computational cost

For a 3202-pixel plate repeatedly reduced until its shorter side reaches approximately 100 pixels, the pyramid evaluates 1,086 local shift candidates per channel instead of 130,321 full-resolution candidates. That is roughly 120 times fewer candidate evaluations, before considering that the early comparisons run on much smaller images.

Using raw-intensity NCC with `min_size = 128`, all 14 provided TIFF plates processed in 47 seconds.

Plate	Green offset	Red offset	Time (s)
cathedral	(5, 2)	(12, 3)	0.2
monastery	(-3, 2)	(3, 2)	0.2
tobolsk	(3, 3)	(6, 3)	0.2
church	(25, 4)	(58, -4)	3.6
emir — fails	(49, 24)	(416, -456)	3.9
harvesters	(60, 17)	(124, 13)	3.7
icon	(41, 17)	(89, 23)	3.9
ilemselga	(40, 7)	(130, 11)	4.0
melons	(82, 11)	(178, 13)	4.0
religious painting	(28, 3)	(68, 7)	4.0
self portrait	(79, 29)	(176, 37)	4.0

Plate	Green offset	Red offset	Time (s)
siren	(49, -6)	(96, -25)	4.0
three generations	(53, 14)	(112, 11)	3.9
wharf	(15, -7)	(83, -16)	3.9

Table 1. Pyramid alignment using raw pixel intensities, NCC, and `min_size = 128`. Thirteen of the fourteen supplied TIFF plates register correctly.

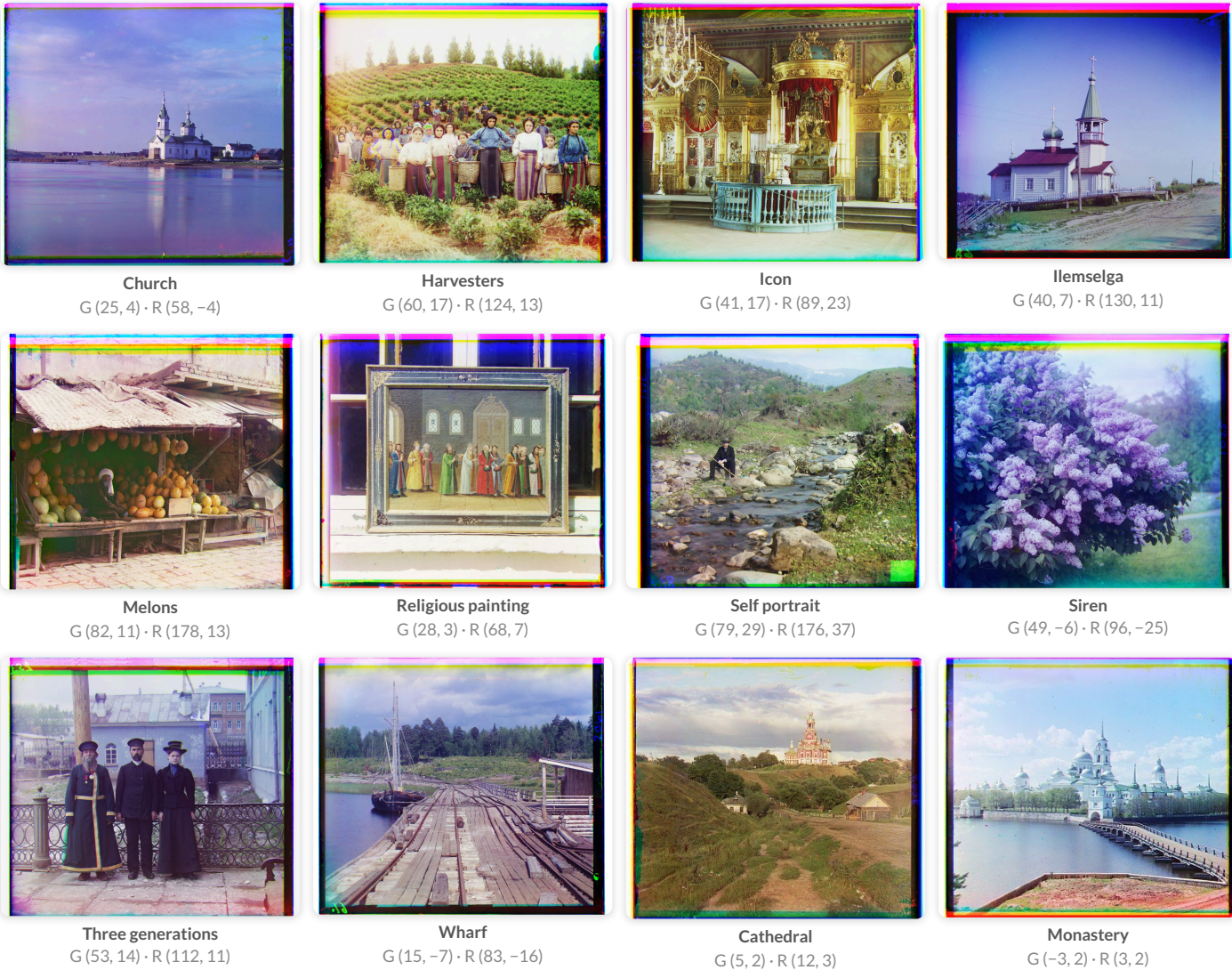


Figure 2. The 13 plates that align correctly with raw-intensity matching. Remaining color fringing is largely confined to the damaged plate boundaries.

03 · Where raw intensity fails

Emir: the issue is not NCC alone

The *emir* plate fails dramatically in the red channel. Raw NCC returns

$$(416, -456)$$

rather than the visually correct displacement, approximately

$$(107, 40).$$

The cause is visible in the emir's robe.

Region	Blue	Green	Red
Robe	0.562	0.385	0.310
Whole plate	0.529	0.489	0.588

Table 2. The robe is brightest in blue and darkest in red, while the whole plate follows the opposite ordering: red is the brightest channel overall.

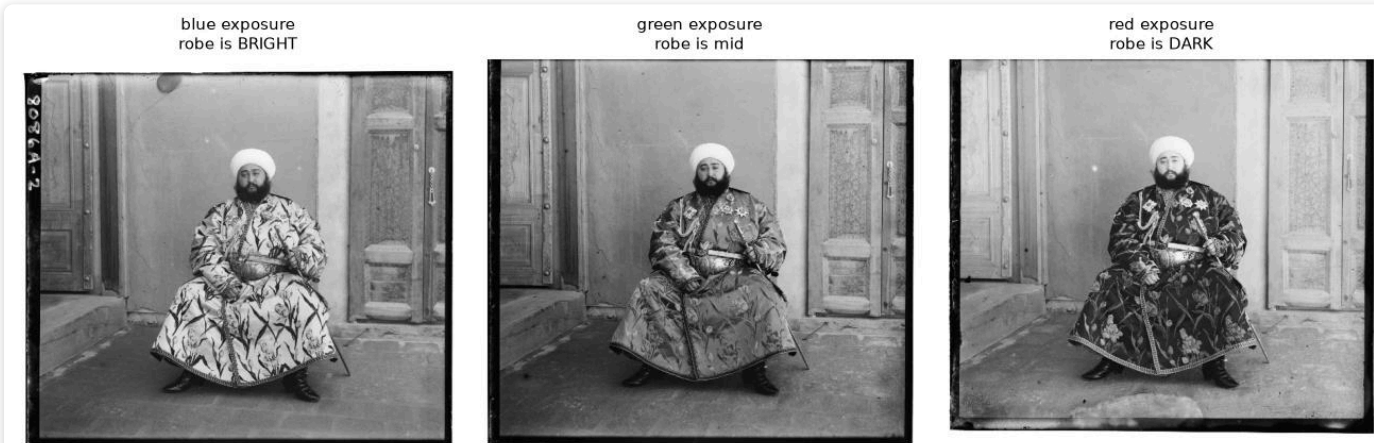


Figure 3. The robe across the blue, green, and red exposures. It is bright in blue, intermediate in green, and nearly black in red.

The robe is inverted relative to the rest of the scene. A matcher based on brightness can therefore prefer an unrelated patch with a similar red intensity distribution over the robe’s true position.

I initially suspected that NCC was simply the wrong metric, so I tested SSD as well.

Emir red-channel method	Offset	Correct?
Raw intensity + NCC	(416, -456)	No
Raw intensity + SSD	(99, -211)	No
Sobel gradient magnitude + NCC	(107, 40)	Yes

Table 3. Both raw-intensity metrics fail, though they fail at different offsets. The green channel consistently recovers (49, 24).



Figure 4. Two raw-intensity metrics produce two different incorrect registrations; gradient matching produces the correct result. If NCC itself were the problem, SSD should have recovered the correct displacement. It did not.

Why NCC does not solve this case

NCC is invariant to a global relationship of the form

$I_2 = aI_1 + b.$

That model does not describe *emir*. The robe changes relative brightness across channels differently from the rest of the image, so no single global a and b can map the blue exposure to the red one.

The problem is not a uniform brightness shift or contrast change. It is a spatially varying photometric change caused by the historical color filters.

Matching gradient magnitude

The final registration method matches Sobel gradient magnitude:

$$M = \sqrt{G_x^2 + G_y^2}.$$

A Sobel response changes sign when contrast reverses, but gradient magnitude does not:

$$(-G_x)^2 = G_x^2, \quad (-G_y)^2 = G_y^2.$$

An edge therefore produces the same magnitude whether brightness rises or falls across it. The robe boundary remains visible in every channel despite its red-blue contrast reversal.

Gradient matching does not require corresponding pixels to have the same brightness. It requires them to preserve the same spatial structure.

The failure is also asymmetric: green aligns correctly under every tested method because its appearance still tracks blue. Only red fails because only red inverts the robe’s contrast. A geometric bug would be expected to affect both channels.

Icon: the coarsest level can be too coarse

Gradient matching fixes *emir*, but it initially broke *icon*, which had registered correctly with raw intensities.

With `min_size = 64`, the gradient-based pyramid returned

$$(862, 128)$$

for the red channel instead of the expected result near

$$(89, 23).$$

Icon contains repeated gilded ornament. At a 64-pixel base level, that ornament collapses into a nearly periodic texture. Several displacements score similarly, and the coarse search locks onto the wrong repeated feature. Higher-resolution refinement cannot escape once the pyramid begins in the wrong displacement basin.

Increasing the base-case size to 128 pixels preserves enough distinctive image structure to resolve the ambiguity.

Base-case size	Red offset	Result
<code>min_size = 64</code>	(862, 128)	Wrong repeated feature
<code>min_size = 128</code>	(90, 23)	Correct



Figure 5. Gradient alignment on icon. At 64 pixels, the search locks onto the wrong repeated ornament. At 128 pixels, the coarsest level retains enough distinctive structure to identify the correct displacement.

This is a useful counterexample to the idea that a smaller base image is always better. The coarsest level is where the algorithm chooses its global displacement basin, so it must still retain enough unique scene information to make that decision.

With gradient magnitude matching and `min_size = 128` , all 14 TIFF plates align correctly.

Plate	Green offset	Red offset	Time (s)
cathedral	(5, 2)	(12, 3)	0.1
monastery	(−3, 2)	(3, 2)	0.1
tobolsk	(3, 3)	(6, 3)	0.1
church	(25, 4)	(58, −4)	3.7
emir	(49, 24)	(107, 40)	3.5
harvesters	(60, 17)	(124, 14)	3.7
icon	(42, 17)	(90, 23)	3.8
ilemselga	(40, 7)	(131, 11)	3.7
melons	(80, 10)	(177, 13)	3.7
religious painting	(30, 7)	(70, 6)	3.6
self portrait	(78, 29)	(176, 37)	3.7
siren	(49, −6)	(96, −24)	3.6
three generations	(54, 12)	(111, 9)	3.5
wharf	(15, −7)	(83, −16)	3.5

Table 4. Pyramid alignment using Sobel gradient magnitude and `min_size = 128` . All 14 provided TIFF plates register correctly.

04 • Four additional Library of Congress plates

I selected four additional plates from the Library of Congress collection and processed them using the same code and fixed parameter set.

These are raw `LC-DIG-prok-0xxxx` glass-negative scans, not the Library's pre-aligned color composites. Aligning the composites would not meaningfully test the registration method.

The selected scans are approximately 1024 pixels tall, so each exposure is only about 340 pixels high. The pyramid therefore runs for only two levels. These images still test the full registration and enhancement pipeline, but they exercise the multiscale search less heavily than the supplied TIFF plates.

Plate	Identifier	Green offset	Red offset
Suna River, Girvas Falls	prok-00264	(2, -1)	(10, -2)
Gagarin family grave	prok-01211	(2, 2)	(8, 3)
Locomotive "A6 132"	prok-00458	(4, 1)	(9, 3)
Tipy Dagestana	prok-01477	(1, 0)	(9, 0)



Figure 6. Additional Library of Congress plates processed with the same registration and enhancement settings.

- **Suna River, Girvas Falls.** The rocks and treeline align cleanly. The rapids do not because the water moved between exposures. The remaining shimmer is subject motion, not a registration error.
- **Gagarin family grave.** This is the weakest result in the additional set. The plate is heavily faded, and gray-world correction introduces a green-magenta cast because the scene is not neutral on average.
- **Locomotive "A6 132."** The locomotive provides the sharpest visual registration check. The number plate remains legible and the red wheels align cleanly, which is notable because red-channel mismatch caused the *emir* failure.
- **Tipy Dagestana.** The shifts are almost entirely vertical, consistent with a plate advanced vertically between exposures.

05 • Enhancements

The final processing pipeline uses one fixed order and one parameter set for every plate:

gradient registration → automatic crop → gray-world white balance → HSV saturation adjustment → percentile contrast.

Each before-and-after comparison changes only the stage being discussed. This makes the contribution of each enhancement easier to isolate.

1. Gradient-based registration

The registration method matches Sobel gradient magnitude rather than raw intensity. This discards edge polarity while preserving edge location and scene structure.

For *emir*, the red displacement changes from

$$(416, -456)$$

to

The remaining plates move only slightly under gradient matching. Among plates that were already correctly aligned with raw intensity, the largest shift change is four pixels.

2. Automatic cropping

Even after registration, image borders remain unreliable. They contain dark plate edges, scratches, damaged regions, and wrapped image strips introduced by circular shifts. These regions create the green and magenta bands visible near the boundaries of several baseline results.

A fixed crop margin was not allowed, so I estimate the border automatically.

For each row or column, I combine two signals:

1. The distance between its mean intensity and the image median.
2. The disagreement among the three aligned channels.

A dark plate edge increases the first signal; a color fringe increases the second.

The threshold is based on the median and median absolute deviation of the interior region:

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|).$$

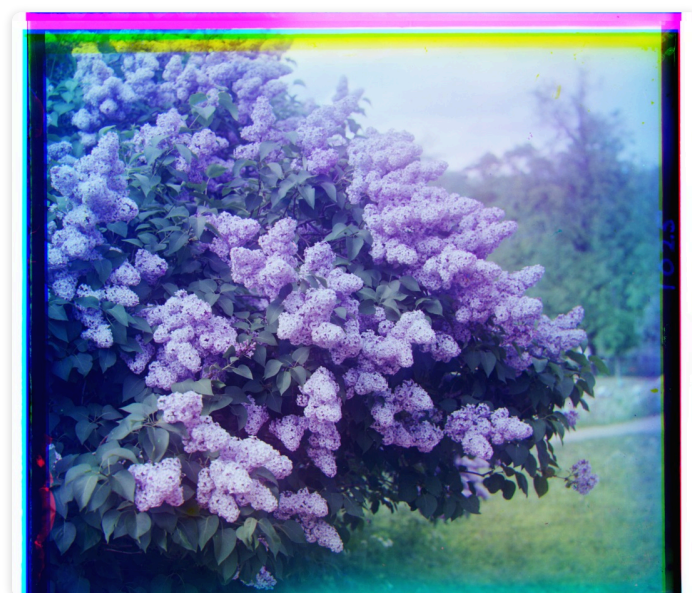
This lets the crop adapt to the noise level and border severity of each plate. The search is also limited to the outer 15% of each dimension, so a dark object or shadow inside the image cannot trigger a runaway crop.

A cropping bug

My first version selected the last row or column whose score exceeded the threshold anywhere inside the search window. A single dark band within the image could therefore drag the crop inward too far.

That version removed 29.9% of *religious painting's* height and 29.9% of *icon's* width.

The corrected method treats a border as a contiguous abnormal region beginning at the edge. It walks inward from each border and stops once the score settles. After that change, the crop removed 6.8% of *religious painting's* height and 9.4% of *icon's* width.



Siren — before



After — crop only
-19.7% rows, -15.8% cols



Figure 7. Automatic crop only. Registration, white balance, color mapping, and contrast remain unchanged. Both plates carry colored bands on all four edges before cropping — siren also retains the dark plate edge and the handwritten “1025” on the right margin. The detector removes them without cutting into the scene.

Gradually fading borders remain difficult because they lack a sharp transition. A top band survives on *harvesters* and *melons* in Figure 12.

3. Automatic contrast

The contrast stage maps a lower percentile to 0 and an upper percentile to 1:

$$I' = \text{clip} \left(\frac{I - p_{\text{low}}}{p_{\text{high}} - p_{\text{low}}}, 0, 1 \right).$$

I compute the percentiles jointly across all three channels rather than independently. This prevents contrast adjustment from functioning as an accidental white-balance operation.

Percentiles are more useful than raw minima and maxima because those values are determined by only one pixel each. A small number of scratches, clipped pixels, or digitization artifacts can occupy the full numerical range even when most of the image remains compressed.

For *church*, the cropped image already spans 0.004 to 1.000. A min-max stretch therefore changes almost nothing, despite the fact that most pixel values lie in a much narrower range.

Church, cropped	Min / max	Standard deviation	Pixels changed
No stretch	0.004 / 1.000	0.176	—
Min/max stretch	0.000 / 1.000	0.177	0.0%
1st/99th percentile stretch	0.000 / 1.000	0.230	96.2%

Table 5. Two extreme pixels make min-max stretching nearly a no-op. Percentile stretching changes 96.2% of pixels and raises the image standard deviation by 31%.

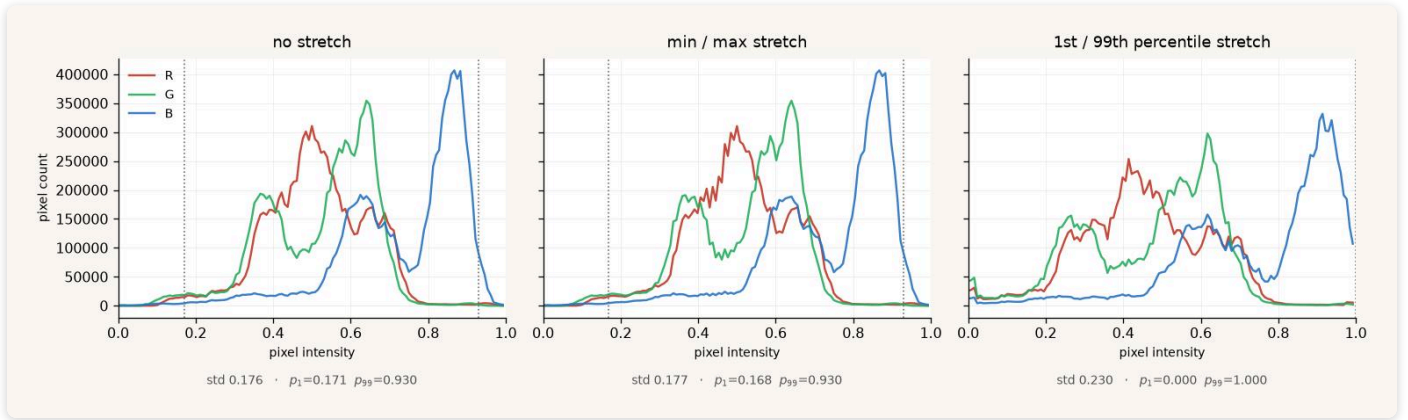


Figure 8. Per-channel intensity histograms for church. The no-stretch and min-max distributions are almost identical because a few pixels already span the available range. Percentile stretching redistributes the bulk of the image across $[0, 1]$, while the clipped tails accumulate at the endpoints.

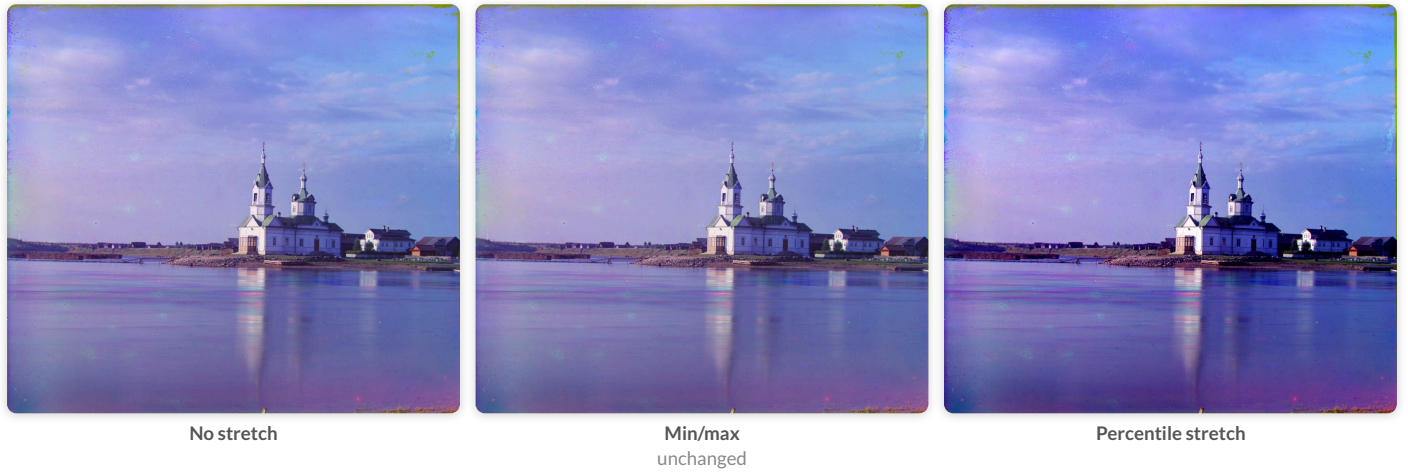


Figure 9. Contrast-only comparison on church. Min-max stretching is visually unchanged; percentile stretching expands the visible tonal range.

Z-score normalization is identical here

I also implemented a z-score version of percentile contrast normalization, expecting it to behave differently. It did not: both versions produced pixel-identical output.

For

$$z = \frac{x - \mu}{\sigma},$$

the percentiles transform as

$$p_z = \frac{p_x - \mu}{\sigma}.$$

Substituting these values into the contrast-stretch expression causes μ and σ to cancel. Under this pipeline, percentile stretching before and after z-score normalization are mathematically equivalent.

4. Automatic white balance

Gray-world white balance assumes that the average scene color should be neutral. It rescales each channel so that all three channel means match:

$$I'_c = I_c \frac{\bar{I}}{\mu_c},$$

where μ_c is the mean intensity of channel c , and $\bar{I}$ is the average of the three channel means.

Gray-world correction reduces the dominant color cast on many plates without requiring manual tuning.

I also tested white-world correction. Applied after percentile contrast stretching, it becomes an exact identity: contrast stretching ensures that every channel reaches 1, so all channel maxima are already equal by construction.

Cathedral	Red	Green	Blue	Channel spread
Cropped	0.556	0.501	0.377	0.179
Contrast only	0.569	0.502	0.351	0.218
Contrast → white world	0.569	0.502	0.351	0.218
White world → contrast	0.565	0.498	0.363	0.201
Gray world → contrast	0.419	0.418	0.413	0.005

Table 6. White-world correction after contrast stretching is exactly an identity. Channel spread is $\max(\mu_R, \mu_G, \mu_B) - \min(\mu_R, \mu_G, \mu_B)$. This metric is not a neutral image-quality measure because minimizing it is gray-world’s own objective, but it clearly demonstrates the white-world identity.



Figure 10. Gray-world correction only on harvesters. The shift is most visible in the clothing.

Gray world fails when a scene is genuinely non-neutral on average. The Gagarin grave plate is the clearest example.

5. HSV color mapping

The Prokudin-Gorskii filters do not correspond exactly to modern sRGB primaries. Even a correctly aligned RGB stack can therefore appear muted.

To improve saturation without changing hue directly, I convert the image to HSV and rescale saturation toward a target value:

$$S' = \text{clip} \left(S \frac{S_{\text{target}}}{P_{75}(S)}, 0, 1 \right),$$

where $P_{75}(S)$ is the image’s 75th-percentile saturation.

The clamp prevents already vivid images from becoming oversaturated and prevents low-saturation images from being stretched implausibly. Hue and value remain unchanged, so the method changes color strength rather than color identity.



Figure 11. Saturation-only comparison on melons.

This is not a true spectral correction. If the historical filters render a red object as slightly orange, saturation scaling will produce a more saturated orange rather than restoring the original red. A channel-mixing matrix could potentially adjust hue, but it would require calibration coefficients from a known color target, which these plates do not provide.

The saturation target and clamp limits were selected visually. This is the least principled part of the pipeline.

06 · All eighteen plates

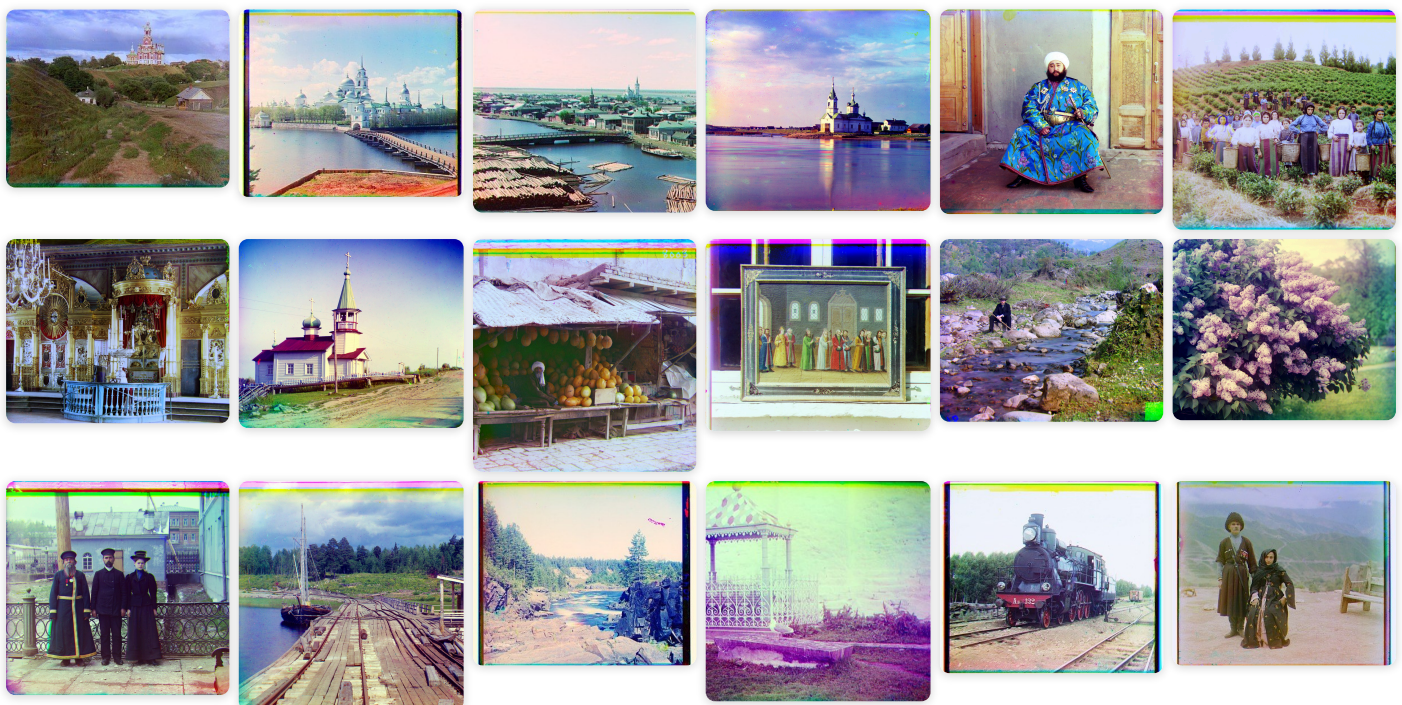


Figure 12. All 18 plates after the complete pipeline.

Registration looks correct on all eighteen plates. The remaining issues are photometric or geometric rather than translational registration errors:

- A top border remains on *harvesters* and *melons*.
- *Three generations* retains faint fringing around some high-contrast edges.
- The Gagarin grave plate remains green-magenta because gray-world white balance is poorly matched to its scene statistics.

07 · Remaining limitations

A global translation is not always enough

Every method in this project applies one displacement vector,

$$(\Delta y, \Delta x),$$

to the entire image.

That assumption fails when different objects move between the three sequential exposures. It can also fail when plate motion, optical distortion, or scene geometry produce local rather than global misalignment.

Three generations still shows faint color fringing along some high-contrast edges. In *suna_river* and *harvesters*, moving water or subjects create differences that no global translation can correct.

Potential extensions include per-region registration, local displacement fields such as optical flow, or searches over scale and rotation in addition to translation. I implemented none of these.

Cropping remains approximate

The automatic crop removes most major border artifacts, but it is not pixel-exact. Gradual border fading remains difficult because there is no strong transition for the detector to identify.

Gray-world white balance can be wrong

Gray-world assumes that the average scene color should be neutral. That is a poor assumption for scenes dominated by pale sky, grass, snow, stone, water, or a large colored object. The Gagarin grave plate makes this limitation especially visible.

Contrast has limited effect on some plates

The percentile stretch has little effect on the four additional Library of Congress scans because they already span most of the available numerical range.

- *Suna River*: standard deviation changes from 0.270 to 0.274.
- *Locomotive*: standard deviation remains 0.312.
- *Gagarin grave*: standard deviation falls from 0.287 to 0.235.

This is the same outlier problem seen in the min-max comparison, but here it affects the input itself: a few extreme pixels can make an image appear to span the full range even when much of its meaningful content does not.

`min_size = 128` comes from one failure case

I selected `min_size = 128` because it corrected *icon*, then applied it globally to satisfy the fixed-parameter requirement. I did not sweep the parameter across the entire collection, so I cannot yet show whether 128 lies on a broad performance plateau or simply works well for this one plate.

Finally, “correct registration” is based on visual inspection. These historical plates do not provide ground-truth channel displacements.

08 • Implementation

The project uses NumPy throughout, with scikit-image used only for:

- Image loading and saving
- Gaussian filtering
- Sobel filtering
- RGB/HSV conversion

At load time, every plate is converted to floating point and split into three equal sections along the row axis. The channels appear top-to-bottom as blue, green, and red; they are not stored in RGB order.

Blue remains the fixed reference. Green and red are aligned independently to blue.

Similarity scoring uses a 10% interior crop. At full resolution, this margin is much larger than the local refinement window. At the coarsest pyramid level, however, the crop is only about 10–13 pixels wide, comparable to the search radius. Some extreme candidates at that level can therefore include a small number of wrapped pixels.

That is a real gap between the intended boundary handling and the current implementation.

Each function was tested against synthetic cases with known ground-truth shifts. All reported offsets, timings, tables, and output images come from one end-to-end run of the final pipeline.

09 • References

Prokudin-Gorskii Collection, Library of Congress, Prints & Photographs Division. [Library of Congress collection](#). Additional plates: `prok-00264` , `prok-01211` , `prok-00458` , and `prok-01477` .

Efros, A. A. *Programming Project #1: Images of the Russian Empire*. CS180/280A, UC Berkeley, Fall 2026.

Szeliski, R. *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.

Photographs by Sergei Prokudin-Gorskii are held by the Library of Congress and have no known restrictions on publication. Images on this page are resized for the web; all registration and processing were performed at full resolution.

[← Back to portfolio](#)

Website template adapted from [Jon Barron](#).